

Deployment Home Task

This document provides a compact summary of the completed Deployment Home Task implementation

by Bisera Naumovska

Deployment Home Task

Technical documentation for the infrastructure assignment, including scope, implementation summary, architecture flow, usage, validation steps, and suggested improvements.

1. Task Scope

The assignment required a monitored web and database stack deployed with Docker Compose, automated with Ansible, and complemented by Kubernetes manifests. Bonus features included Helm, Terraform, CI validation, alerting, and automated database backup/restore.

Main Technologies

- Nginx for HTTP/HTTPS web serving
- PostgreSQL for structured data storage
- Prometheus + Node Exporter for metricsBlackbox Exporter for website availability checks
- Loki + Promtail for explicit log monitoring
- Grafana for dashboards and log exploration
- Ansible for deployment and SSH hardening
- Alertmanager for email notifications

Delivered Extras

- Kubernetes Deployment + Service
- Helm chart with Ingress and HPA
- Terraform local VPS-style provisioning
- GitHub Actions validation workflow
- Database backup and restore automation

2. Requirement Coverage

Core Requirements

Requirement	Status	Implementation	Evidence
Ansible deploy for monitoring and scalable targeting	Done	Playbook deploys stack group targeting	ansible/playbook.yml, ansible/inventory/hosts.ini
Monitor logs + CPU + RAM + disk	Done	CPU/RAM/disk alerts via Prometheus; logs via Promtail to Loki	docker-compose/prometheus/alert.rules.yml, docker-compose/loki/loki-config.yml, docker-compose/promtail/promtail-config.yml
Web server with specific HTTP and HTTPS content	Done	Nginx serves separate HTTP and HTTPS page roots	docker-compose/nginx/nginx.conf, docker-compose/html-http/index.html, docker-compose/html-https/index.html
Restrict HTTP by IP allowlist	Done	allow/deny rules in Nginx HTTP server block	docker-compose/nginx/nginx.conf
SSL enabled	Done	TLS cert/key configured in Nginx HTTPS server block	docker-compose/nginx/nginx.conf
Password auth disabled SSH key-only security	Done	Done and key-based flow configured	ansible/playbook.yml, terraform/main.tf
Web availability monitored	Done	Blackbox HTTP/HTTPS probes + WebserverDown alert rule	docker-compose/prometheus/prometheus.yml, docker-compose/prometheus/alert.rules.yml
DB setup with required table and at least 30 rows	Done	SQL init script creates table and inserts 30 records	docker-compose/init-db/01-init.sql
Everything in Docker Compose	Done	Full stack is declared in one Compose file	docker-compose/docker-compose.yml
Basic Kubernetes Deployment + Service	Done	Plain kubectl manifests for Deployment and Service	k8s/deployment.yaml, k8s/service.yaml

Deliverables

Deliverable	Status	Evidence
Ansible playbook	Done	ansible/playbook.yml
Kubernetes manifests (Deployment/Service)	Done	k8s/deployment.yaml, k8s/service.yaml
Architecture/design documentation	Done	README.md (architecture, assumptions, limitations)
Brief report with challenges	Done	README.md (work summary and challenges)
VPS access details	Done	README.md (VPS section), terraform/main.tf

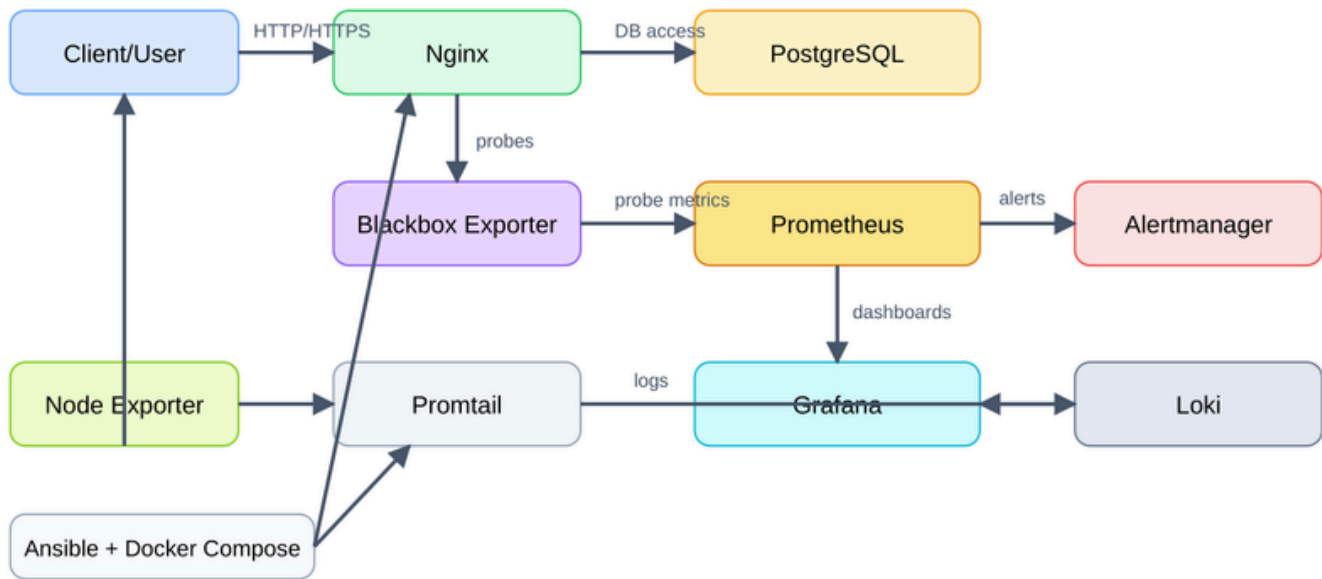
Bonus Points

Bonus Item	Status	Evidence
Helm + Ingress TLS + HPA	Done	helm/webserver-chart/templates/ingress.yaml, helm/webserver-chart/templates/hpa.yaml
VPS provisioning via Terraform	Done	terraform/main.tf
CI validation pipeline	Done	K8s and Ansible syntax checks are present; ansible-lint install exists
Alerting for resource thresholds	Done	docker-compose/prometheus/alert.rules.yaml, docker-compose/alertmanager/alertmanager.yaml
Automated DB backups	Done	docker-compose/backup.sh, .github/workflows/db-backup.yml

3. Implementation steps

1. Created Docker Compose stack for PostgreSQL and Nginx.
2. Added SQL initialization for the active_network_equipment table and mock seed data.
3. Configured Nginx for HTTP, HTTPS, IP restrictions, and SSL certificates.
4. Added Prometheus and Node Exporter for CPU, RAM, and disk monitoring.
5. Added Alertmanager and alert rules via email
6. Added Blackbox Exporter to probe HTTP and HTTPS endpoints.
7. Added Loki and Promtail for minimal explicit log monitoring.
8. Connected Grafana to Prometheus and Loki for metrics and log visualization.
9. Created Ansible playbook and roles for deployment and SSH hardening.
10. Added Kubernetes manifests, Helm chart, Terraform provisioning, and CI workflows.

4. Architecture and Communication Diagram



Metrics path: Node Exporter → Prometheus → Grafana/Alertmanager. Availability path: Blackbox → Prometheus. Log path: Promtail → Loki → Grafana

5. How to Run and Validate the Solution

Check	Command	Expected Result
Ansible syntax	<code>ansible-playbook -i ansible/inventory/hosts.ini ansible/playbook.yml --syntax-check</code>	Playbook parses successfully
Compose services	<code>docker compose --env-file .env -f docker-compose/docker-compose.yml ps</code>	Services show status Up
HTTP endpoint	<code>curl -I http://localhost:8080</code>	HTTP 200 response
HTTPS endpoint	<code>curl -k -I https://localhost:8443</code>	HTTP 200 response
Database seed	<code>SELECT COUNT(*) FROM active_network_equipment</code>	At least 30 rows
Metrics targets	<code>curl -s http://localhost:9090/api/v1/targets</code>	node_exporter, blackbox_http, blackbox_https are up
Availability probe	<code>curl -s 'http://localhost:9090/api/v1/query?query=probe_success'</code>	Value 1 for HTTP and HTTPS
Logs ingestion	<code>curl -sG 'http://localhost:3100/loki/api/v1/query' --data-urlencode 'query={job=\"docker-logs\"}'</code>	Status success and returned log streams
Kubernetes validation	<code>kubectl apply --dry-run=client -f k8s/</code>	Deployment and Service validate successfully

6. VPS Access Details

Local Terraform-provisioned VPS-like container:

- SSH: localhost:22223
- HTTP: localhost:8082
- HTTPS: localhost:8444
- Grafana: localhost:3002
- Prometheus: localhost:9092

Verified SSH command: `ssh -i ~/.ssh/id_rsa -p 22223 root@localhost`

Note: Key provisioning for this local demo assumes `~/.ssh/id_rsa.pub` exists on the host machine.

For a real remote VPS, replace localhost with the server public IP and set host entries in `ansible/inventory/hosts.ini`.`

7. GitHub Actions - workflow pipelines

The complete solution is published on GitHub under the repository name **deployment_home_task**, inside the account BiseraNaumovska.

You can access it directly through this URL:

https://github.com/BiseraNaumovska/deployment_home_task

This repository contains the full implementation of the home task, including Ansible automation, Docker Compose services, monitoring and alerting configuration, Kubernetes/Helm manifests, Terraform provisioning files, and GitHub Actions workflows.

For contributors or reviewers who want to download the project locally, the clone URL is:

https://github.com/BiseraNaumovska/deployment_home_task.git

Database BACKUP

This workflow is manual (with an optional daily cron already present but commented out). It starts only PostgreSQL from your compose stack, waits until the database is ready, runs the backup script (full or schema_only), and uploads the backup as a workflow artifact. To run it, go to Actions, select Database BACKUP, click Run workflow, choose backup_type, and start the run

The screenshot displays the GitHub Actions interface for the 'Database BACKUP' workflow. The workflow is triggered by 'workflow_dispatch' and consists of a single job named 'database-backup'. The job is currently in a 'Success' state, having completed 18 seconds ago. The workflow file 'db-backup.yml' is shown, and the job details indicate it produced an artifact named 'postgres-backup-33515214758'. A red arrow points to the 'Run workflow' button, and another red arrow points to the 'postgres-backup-33515214758' artifact.

Database BACKUP
db-backup.yml

2 workflow runs

This workflow has a workflow_dispatch event trigger.

Database BACKUP #2: Manually run by BiseraNaumovska

Database BACKUP #1: Manually run by BiseraNaumovska

Use workflow from
Branch: main

Backup type (full or schema_only) *

full

full

schema_only

Run workflow

Database BACKUP #2

Summary

All jobs

database-backup

Run details

Usage

Workflow file

Manually triggered 18 hours ago

BiseraNaumovska

Status: Success

Total duration: 18s

Artifacts: 1

db-backup.yml

on: workflow_dispatch

database-backup 12s

Annotations

1 warning

database-backup

Node.js 20 is deprecated. The following actions target Node.js 20 but are being forced to run on Node.js 24: actions/checkout@v4, actions/upload-artifact@v4. For more information see: <https://github.blog/2024-10-22-nodejs-20-eol/>

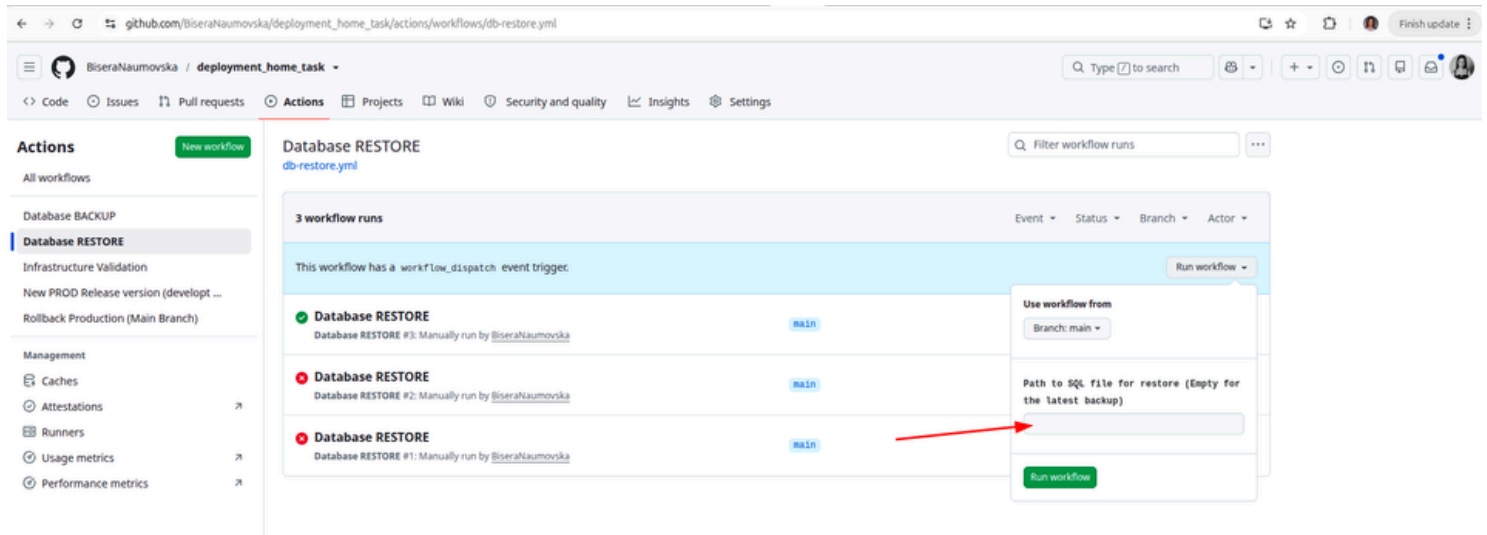
Artifacts

Produced during runtime

Name	Size	Digest
postgres-backup-33515214758	6.4 KB	sha256:85792389bd967334e96a4ceef88baa7e6b3c959dd442f57d24c3c6ad...

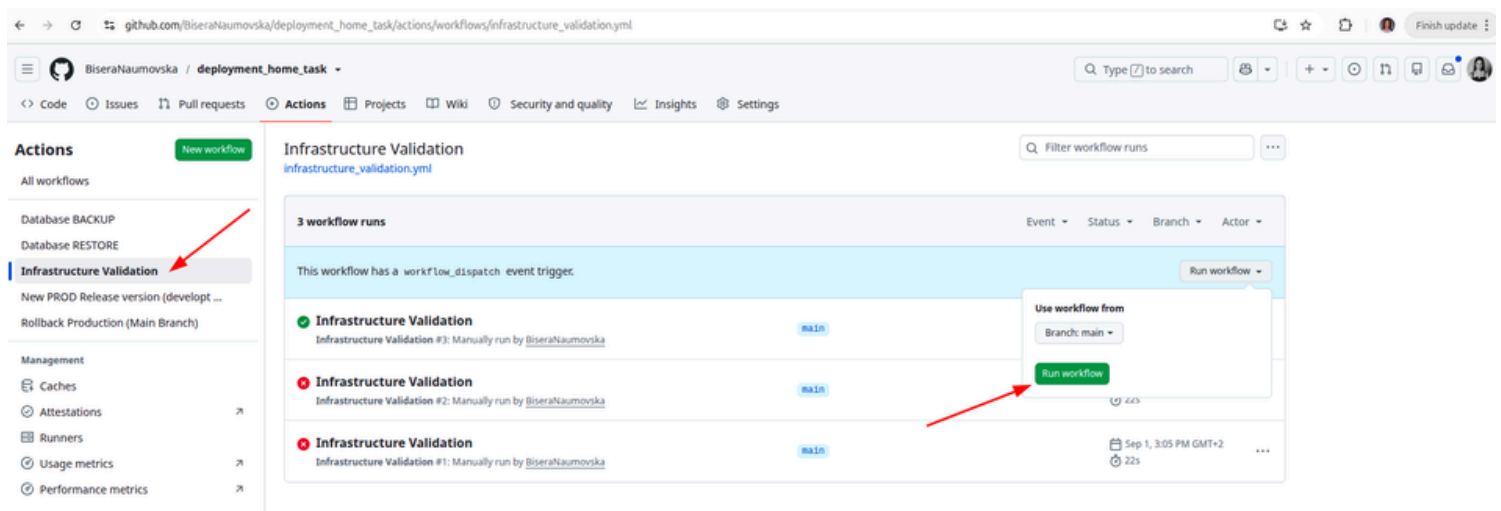
Database RESTORE

This workflow is manual and restores database data from a selected SQL backup file or the latest backup if input is empty. It starts PostgreSQL, prints row count before restore, executes restore, then prints row count after restore for quick verification. To run it, open Actions, choose Database RESTORE, click Run workflow, optionally set backup_file, and run.



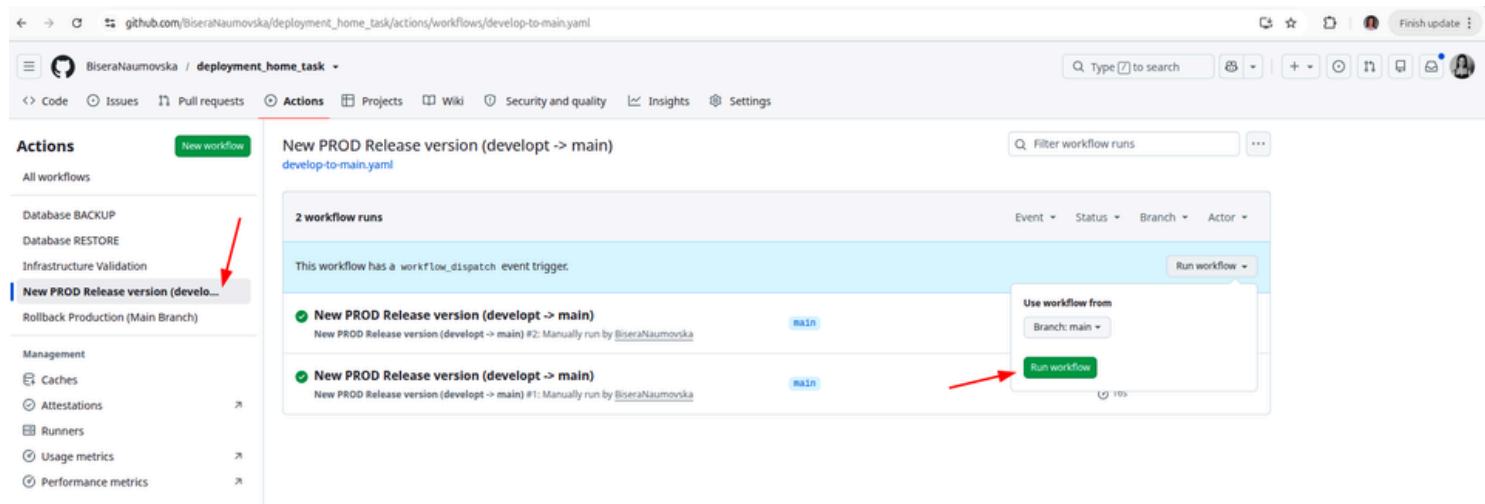
Infrastructure Validation

This workflow is currently manual and is started from GitHub Actions. It checks Ansible playbook syntax, lints the Helm chart, creates a temporary kind cluster, and validates Kubernetes manifests with dry-run apply. To run it, open your repository on GitHub, go to Actions, choose Infrastructure Validation, and click Run workflow.



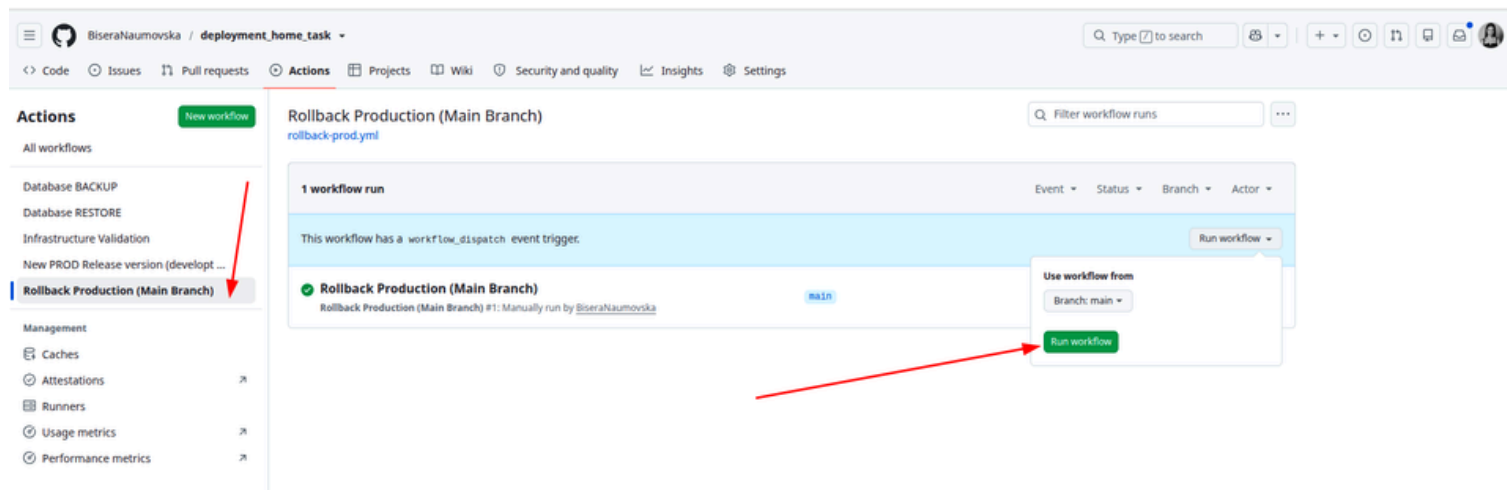
New PROD Release version (develop -> main)

This workflow is manual and promotes develop to main using a fast-forward-only merge, then creates a timestamped production tag. It is meant for controlled production releases when develop is already validated. To run it, open Actions, choose New PROD Release version, click Run workflow, and execute it.



Rollback Production (Main Branch)

This workflow is manual and rolls main back to the previous production tag (the second newest prod.* tag), then force-pushes main. It is intended for emergency rollback when the latest release must be reverted quickly. To run it, go to Actions, select Rollback Production, click Run workflow, and start it.

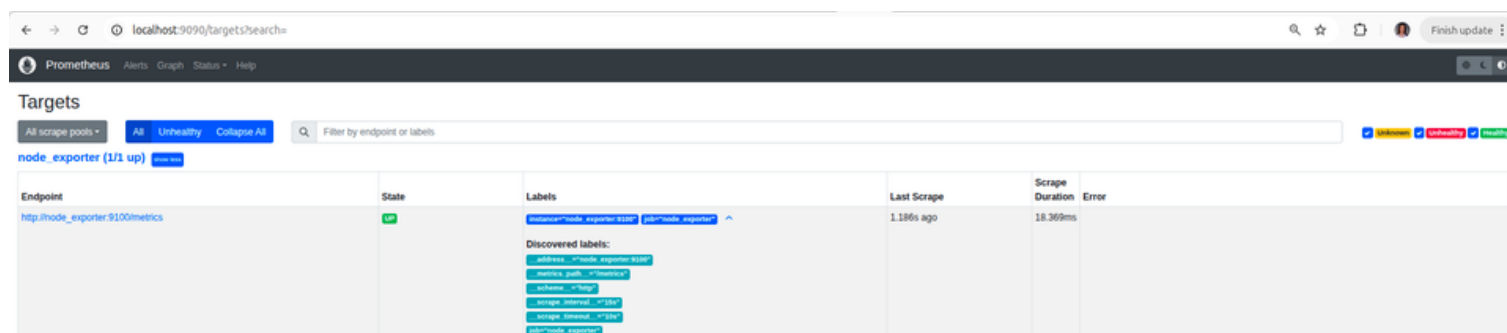


8. Monitoring and Alerting

This project uses a fully open-source observability stack. Each tool has a specific responsibility, and together they provide metrics, logs, dashboards, and notifications.

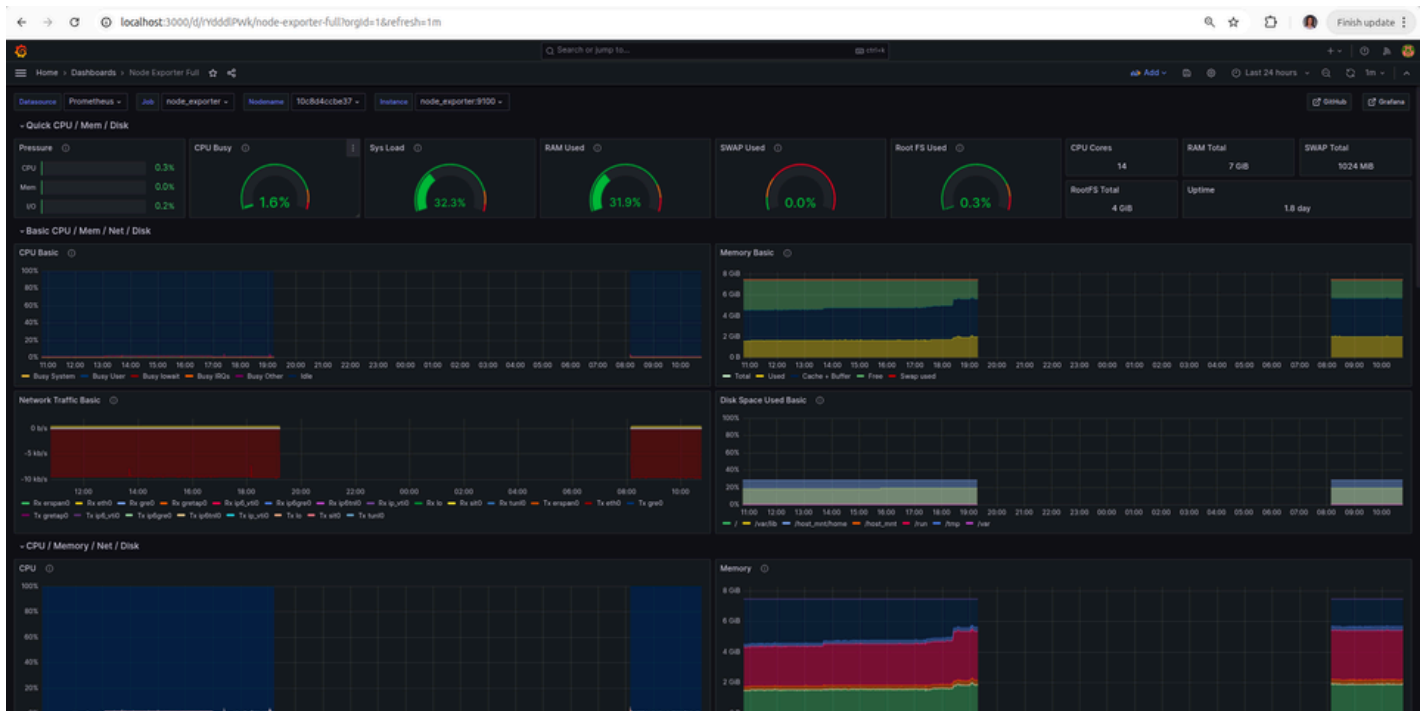
Prometheus (Metrics Collection and Alert Evaluation)

- Prometheus periodically scrapes metrics endpoints from services like `node_exporter` and `blackbox_exporter`.
- It stores time-series data (CPU, memory, disk, probe success, and similar metrics).
- It loads alert rules from `alert.rules.yml` and continuously evaluates conditions such as high CPU usage or web endpoint downtime.
- When a rule is triggered, Prometheus sends the alert event to Alertmanager.



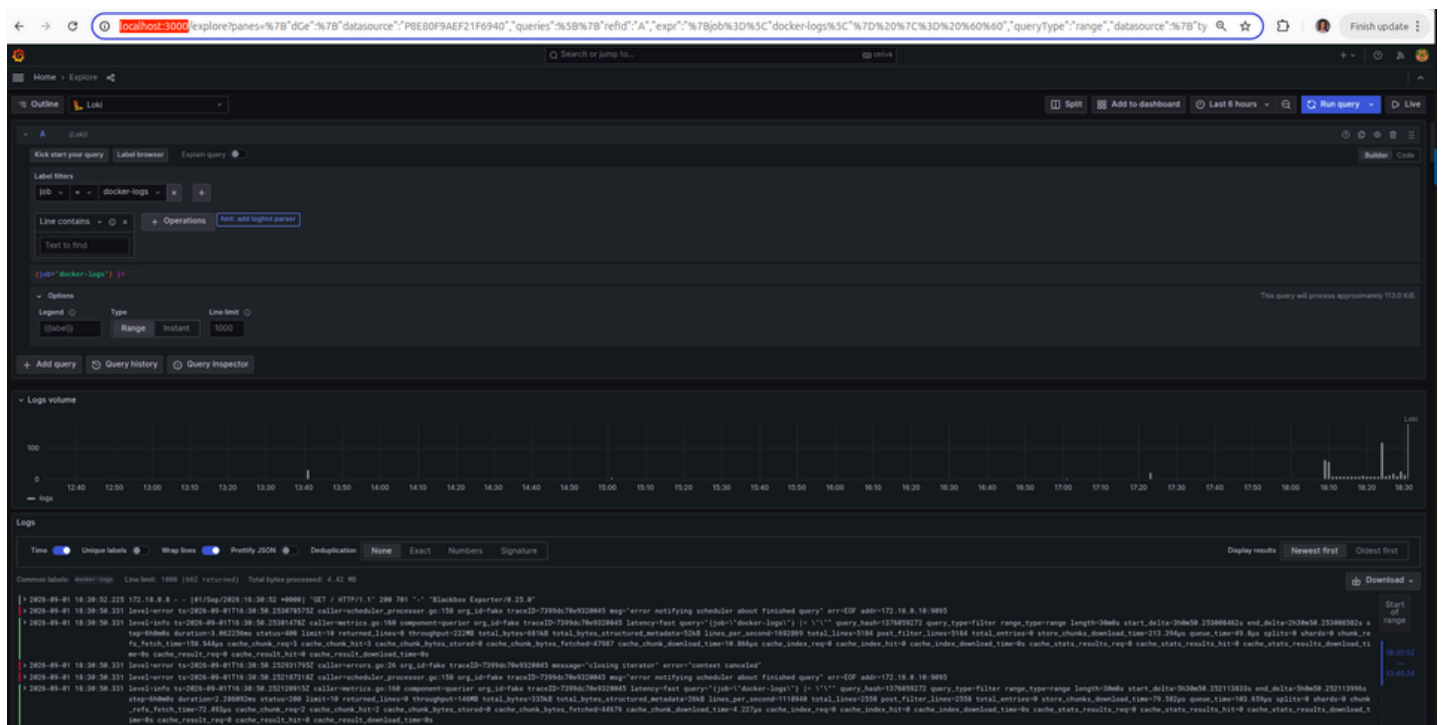
Grafana (Visualization Layer)

- Grafana is the UI layer where operators view dashboards and investigate incidents.
- It does not collect metrics or logs by itself; it reads data from configured datasources.
- In this repository, Grafana connects to Prometheus for metrics and Loki for logs.
- This allows side-by-side investigation: metric spikes and related container logs in one interface.



Loki (Log Storage and Query Engine)

- Loki is a log aggregation backend designed for efficient indexed labels and log search.
- It receives logs from Promtail and stores them for querying.
- Grafana queries Loki to display log streams and filtered log results.

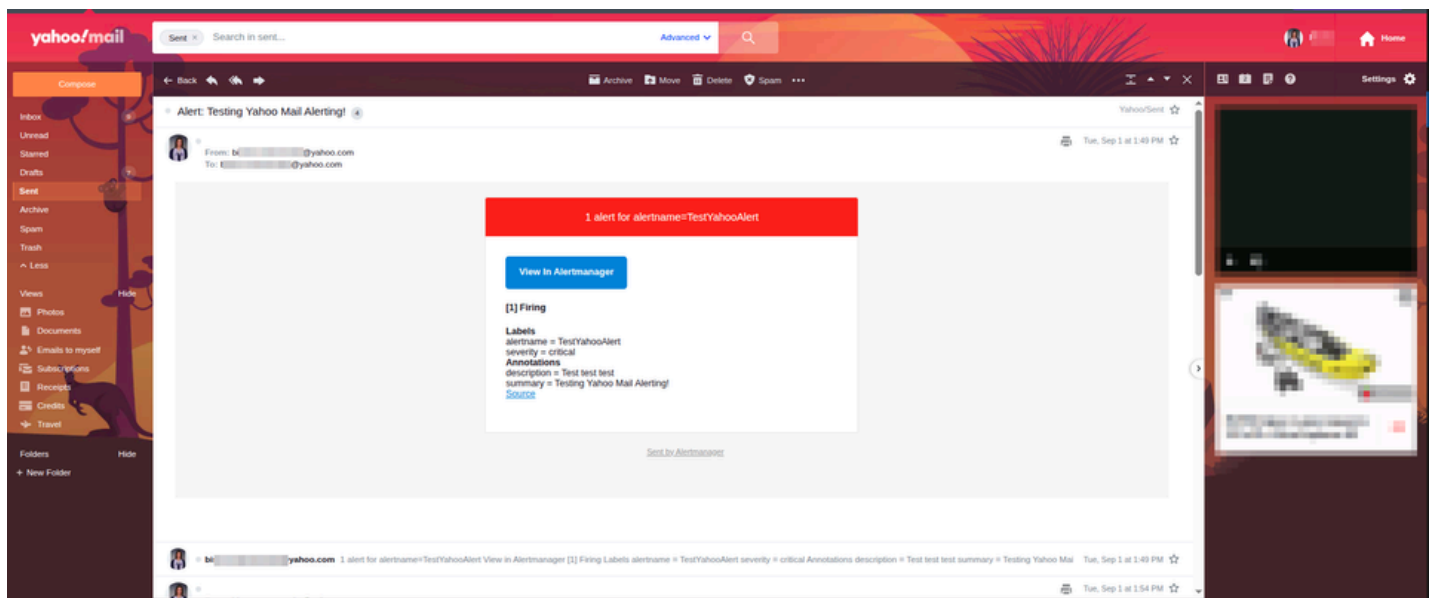
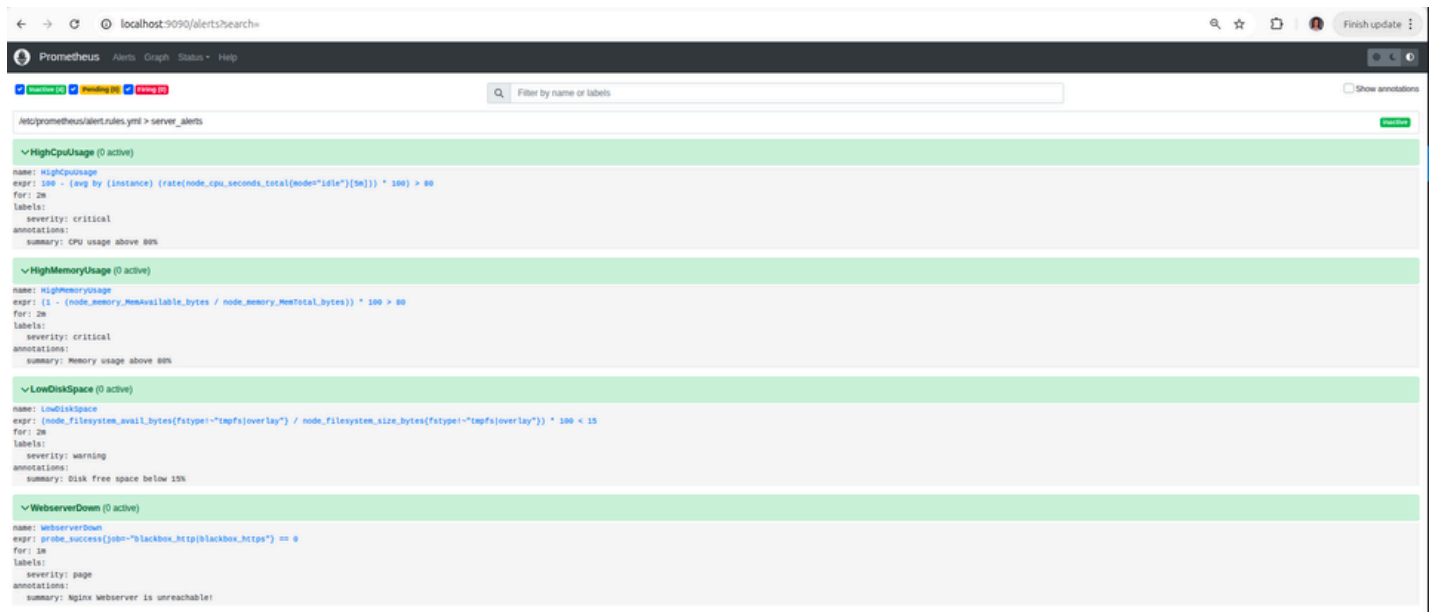


Promtail (Log Collector and Shipper)

- Promtail runs as the log collection agent.
- It reads Docker container log files from the host path mounted into the promtail container.
- It attaches labels (for example job/container labels) and pushes log entries to Loki.

Email Alerting (Alertmanager)

- Alertmanager receives active alerts from Prometheus.
- It groups and routes them using alertmanager.yml.
- The current setup sends notifications through SMTP to email recipients.
- This provides near-real-time operational notification even when no dashboard is being watched.



How They are Connected (End-to-End)

1. Services expose metrics and generate logs.
2. Prometheus scrapes metrics and evaluates alert rules.
3. Promtail collects logs and pushes them to Loki.
4. Grafana reads metrics from Prometheus and logs from Loki for visualization.
5. On alert conditions, Prometheus forwards alerts to Alertmanager.
6. Alertmanager sends email notifications.

9. Challenges

- Keeping balance between required complexity and bonus features
- Coordinating multiple tools (Ansible, Compose, Terraform, K8s, Helm, GitHub Actions)
- Discovering Grafana, Prometheus, Loki and Promtail

10. Future Possible Improvements

- Add a real `ansible-lint` execution step in the infrastructure validation workflow, not only Ansible syntax check.
- Add a `docker compose --env-file .env -f docker-compose/docker-compose.yml config` validation step in CI to verify the main runtime stack configuration.
- Add a lightweight smoke test in the backup and restore workflows to verify DB readiness and expected table/data state.

11. Work Summary

This project was completed over 3 days of focused work.

The implementation and configuration were developed in Visual Studio Code, using a Linux workstation and terminal-based workflow for build, deployment, debugging, and validation tasks.

The final document is prepared in Canva and the diagrams are prepared in draw.io.

Overall, this was a challenging but valuable task because it required combining multiple tools and practices into one complete, working solution.

Completed implementation includes:

- Nginx web server on HTTP and HTTPS with separate HTML pages
- HTTP IP allowlist and TLS setup in Nginx
- PostgreSQL with required active_network_equipment schema and 30 mock records
- Docker Compose stack for application, database, monitoring, and alerting
- Explicit log monitoring with Loki + Promtail integrated into Grafana
- Ansible playbook for SSH hardening, Docker runtime setup, and stack deployment
- Kubernetes basic manifests (Deployment + Service)
- Helm chart with Kubernetes resources
- CI workflows for infrastructure validation, new production version and DB backup/restore automation